Task 2 — From data to AI

Group 7: Bilgin Eren, Alexia Cobzaru, Raluca Costache, Melat Assefa, Mohamed Elshami, Musaed Al-Fareh. Revised 26 October 2026 after lecturer feedback of 1 October (v1 submitted 25 September).

1. Data

The system needs two kinds of data: course descriptions to recommend from, and a skill vocabulary to explain where a course leads. We use the Online Courses catalogue from Kaggle (scraped from Coursera, FutureLearn, Udacity and Simplilearn) and ESCO, the European Commission's taxonomy of skills and occupations, with its ISCO occupation groups. All figures below are written by scripts/profile_data.py, scripts/make_split.py and scripts/data_resources.py.

Volume. The nine CSV files take 52.5 MB on disk and 115.8 MB in memory as pandas DataFrames, so everything fits in RAM on an ordinary laptop (16 GB, CPU only); the largest is ESCO's occupation–skill relations table (126,051 rows, 26.6 MB). Courses: 8,092 rows × 45 columns, reduced to 5,275 × 24 after cleaning. Only the 2,819 Coursera rows carry a category label, and the 11 classes are imbalanced: Business has 895 courses (32%), Math and Logic 22. Skills are listed for 39.7% of cleaned courses (4,229 distinct strings). ESCO: 13,960 skills, cleaned to the 2,154 skills linked to 319 occupations through 8,376 relations; 619 ISCO groups.

Velocity. Both sources are static snapshots (the catalogue was scraped once; ESCO is re-released about yearly), so data is prepared in batch, not streamed. Speed matters when a student asks a question: the SRS requires a shortlist within 3 s (NFR-1) and each AI lookup within 500 ms (NFR-4). Measured with pandas 2.2 on that laptop (median of 5 runs), loading all nine files takes 1.1 s, once at start-up; filtering courses by category takes 1.3 ms, a keyword search over all titles 8.9 ms, and fetching one occupation's skills 1.4 ms, well inside the budget.

Variety. Free text (titles, intros, comma-separated skills), categorical labels (7 of them in Chinese, Japanese, Spanish or Portuguese), numeric fields (ratings, durations) and ESCO's linked tables joined by URI keys.

Veracity. Cross-checks: all 8,092 course URLs point to the domain of the site the row claims; 199 labelled course texts appear more than once and none carries two different categories, so the labels are consistent; every ESCO relation points to an existing skill and occupation, and every occupation to an existing ISCO group. Known weaknesses: FutureLearn's 4,843 rows collapse to 2,031 unique courses, 60% of courses list no skills, and no gold course–ESCO pairs exist.

Preprocessing. We aligned a source side (title + intro; skill string) with a target side (category; ESCO skill), mapped the 7 non-English labels to their English class, dropped 242 labelled rows repeating the same title and intro, kept 24 of 45 columns, and froze a stratified 70/15/15 split (1,805 / 386 / 386, seed 42) guarded by a checksum test.

2. Model

Two models each solve one part of the task: Model 1 sorts a course into a subject so the system can narrow the catalogue; Model 2 maps a course's free-text skills onto ESCO so the system can explain where a path leads.

Model 1: course → category (supervised classification). Input: title + short intro; output: one of 11 categories with a confidence. Three models were compared on dev (Table 1) and the best dev macro-F1 was scored once on test: accuracy 0.795, macro-F1 0.772. Macro-F1 is reported because always guessing Business already gives 32% accuracy. The weakest classes are the smallest (Social Sciences F1 0.58, Personal Development 0.64). The MLP drives its training loss to 0.022 yet has a lower dev macro-F1, so it overfits the 1.8k training texts; we kept the linear model.

Table 1. Model 1 on the dev set (386 courses).

Model	Accuracy	Macro-F1	Training	Inference
Majority-class baseline	0.321	0.044	–	<0.01 ms/item
TF-IDF + Logistic Regression (selected)	**0.806**	**0.752**	**0.62 s**	**0.13 ms/item**
TF-IDF → SVD(300) → MLP(256)	0.811	0.724	7.18 s, 92 epochs	0.09 ms/item

Model 2: course skill → ESCO skill (semantic retrieval). Input: one course skill; output: the top-5 ESCO skills with similarity scores. Nothing is trained on our data: the 2,154 ESCO skills are ranked by cosine similarity with a pretrained sentence encoder (all-MiniLM-L6-v2) or word + character TF-IDF. A lookup table is not enough: exact string matching links only 7.4% of the 4,229 distinct course skills to any of ESCO's 99,624 labels, and 2.4% to our 2,154-skill index. With no labelled course–ESCO pairs, we held out one alternative label per ESCO skill and used it as a query with a known answer (Table 2).

Table 2. Model 2 on held-out ESCO alternative labels (index: 2,154 skills).

Method	Top-1	Top-5	MRR@5
Exact string match (baseline)	0.000	–	–
Word + character TF-IDF cosine	0.816	0.927	0.864
Sentence embeddings (all-MiniLM-L6-v2)	**0.866**	**0.962**	**0.905**

3. Model adequacy

Both are standard machine-learning methods sized to the data. A lookup table is too simple (7.4% coverage), and training or fine-tuning a transformer on 1.8k labelled texts would overfit. Model 1 is closed-set classification on a small labelled set, where a linear model on TF-IDF already matches the MLP. Model 2 has no labelled pairs and must handle phrasings it has never seen, which rules out a supervised classifier and suits a pretrained encoder used only for retrieval. Everything runs on a CPU in seconds.

Generalisation to real data. Table 2 is optimistic: ESCO's alternative labels are written in the same style as the index, while course skills are written by course authors. On the 4,229 real course skills, 65.9% reach a similarity of at least 0.35 (TF-IDF), so about a third have no confident match. The system therefore withholds any match below a threshold chosen on held-out data instead of guessing (SRS FR-13), and we will measure precision on a manually checked sample of real course skills (SRS AIR-2). Model 1 faces a similar shift: it is trained on Coursera only but applied to all four sites, so its low-confidence predictions are marked "uncategorised".

4. Software

Python 3.13 in a conda environment (environment.yml), Windows 11, CPU only, Git and GitHub, seed 42. Data: pandas 2.3 and NumPy 2.3; profiling charts with matplotlib 3.10 and seaborn 0.13. Models: scikit-learn 1.7 (TF-IDF, TruncatedSVD, LogisticRegression, MLPClassifier, majority baseline, all metrics); sentence-transformers 5.5 on PyTorch 2.12 for all-MiniLM-L6-v2; joblib for saved pipelines. Notebooks run headlessly with nbconvert, and every reported number is written to data/processed/ by the script or notebook that produced it.